MEMORIA TECNICA & AUDITORIA DE SISTEMA

ATLAS-SAP OPERACIONES v3.0

Suite Operativa On-Premise para Gestion y Control de RFCs, Solicitudes y Ventanas de Mantenimiento SAP

Organizacion:	EAS Consulting Group S.A. de C.V.
Cliente / Entorno:	Operacion SAP / KOF & EAS Basis Team
Entorno de Ejecucion:	Proxmox VE 8.x (VM 10.10.10.102) - Docker Compose
Fecha de Emision:	05 de Noviembre de 2026
Autor / Supervisor:	Hermes Core / Soporte TI / Enrique Aguilar
Clasificacion:	Documento Tecnico de Arquitectura y Seguridad

ALCANCE DEL DOCUMENTO:
Este documento consolida la arquitectura completa del proyecto ATLAS, la guia metodologica para la separacion modular en dos repositorios de GitHub (Frontend y Backend), y una auditoria granular exhaustiva en cuatro dominios clave de ingenieria: **QA & Testing**, **UX/UI**, **CI/CD** y **Ciberseguridad**, con sus planes de accion y mitigacion correspondientes.

1. Arquitectura Global del Sistema

ATLAS-SAP Operaciones es una solucion corporativa disenada para centralizar el ciclo de vida de los cambios (RFCs), solicitudes operativas y ventanas de ejecucion sobre infraestructuras SAP y No SAP. El sistema opera actualmente bajo una arquitectura de contenedores Docker en un servidor virtualizado Proxmox VE con direccionamiento IP privado 10.10.10.102.

Capa / Componente	Tecnologia Empleada	Rol y Responsabilidad
Frontend SPA	HTML5, CSS3 Nativo, Vanilla JS ES6+	Interfaz de usuario reactiva sin dependencias pesadas de frameworks, optimizada para baja latencia.
Reverse Proxy / SSL	Nginx 1.25 Alpine (Contenedor atlas_nginx)	Terminacion TLS 1.3 en puerto 8443, proxy pass hacia API Express, compresion Gzip y cabeceras de seguridad.
Backend REST API	Node.js v20 LTS / Express (atlas_backend)	Capa de servicios REST emulando PostgREST (/rest/v1), autenticacion JWT (/auth/v1) y motor de permisos.
Base de Datos	PostgreSQL 15 (atlas_postgres)	Almacenamiento relacional transaccional, esquema auditado con triggers de auditoria y claves foraneas estrictas.
Automatizaciones	n8n Workflow Engine (atlas_n8n)	Disparadores de notificaciones programadas, alertas operativas y sincronizaciones en puerto 5678.
Perimetro WAN	FortiGate NGFW + DDNS	Mapeo de puertos seguros y filtrado perimetral hacia easconsulting.fortiddns.com.

2. Desglose Modular de Capacidades de Negocio

La aplicacion se organiza en modulos operativos especializados interconectados a traves de una base de datos PostgreSQL unificada:

A. Gestion Integral de RFCs (Ciclo de Vida de Cambios):

Control estricto de los estados de aprobacion y ejecucion: *En espera de VOBO*, *Aprobado*, *En Ejecucion*, *Finalizado con exito*, *Finalizado sin exito* y *Cancelado*. Soporta planes de retorno de ventana y horarios de ejecucion extendidos.

B. Modulo de Solicitudes Operativas:

Seguimiento de requerimientos con folio identificador unico, fecha de inicio obligatoria con calendario picklist, asignacion de Responsable TQS (picklist dinamico con exclusion de roles genericos), y trazabilidad cronologica de cambios de estatus (estatus_historial).

C. Filtro de Vista de Operacion (Todos vs TQS en Menu Lateral):

Integracion de un selector global en el menu lateral que permite alternar la vista entre **Todos** (vision de control operativo completa) y **TQS** (filtrado instantaneo de RFCs donde el usuario activo figura como propietario, colaborador asignado o responsable del sistema).

D. Ingesta Masiva y Parseo de Correo (.eml / .msg):

Motor de autocompletado desde correos de autorizacion de ventanas, con algoritmos de sanitizacion mojibake para neutralizar corrupciones de caracteres acentuados comunes en exportaciones de Excel de SAP.

3. Estrategia de Separacion en Dos Repositorios GitHub

Conforme a la directiva de arquitectura, el proyecto monolitico se reorganiza en dos carpetas estructurales independientes para ser publicadas en los repositorios designados:

Componente	Repositorio Destino en GitHub	Contenido y Entregables
Frontend	https://github.com/eaguilarh01/atlas-frontend	index.html, styles.css, favicon.png, plantillas Excel de carga masiva, recursos estaticos y configuracion de linters frontend.
Backend	https://github.com/eaguilarh/atlas-backend	server.js, schema_onpremise.sql, scripts/ (seeds, backups, diagnosticos), Dockerfile, docker-compose.yml y pruebas unitarias/integracion.

Respuestas a las Preguntas Estrategicas del Proyecto:

1. ¿El proyecto seguiria trabajando tal como lo estamos haciendo o ya estariamos apuntando a GitHub?
El entorno de produccion On-Premise (Proxmox 10.10.10.102) continuara operando al 100% sin interrupciones. La transicion hacia GitHub se realiza en paralelo: el servidor local pasa a consumir ramas estables (main/master) mediante clones limpios de los dos repositorios, permitiendo seguir trabajando en caliente mientras GitHub funge como fuente de la verdad versionada y respaldo seguro.

2. ¿Cuanto tiempo nos llevaria hacer esa separacion?
La separacion fisica de archivos y creacion de repositorios toma aproximadamente **2 a 3 horas**. La configuracion de pipelines CI/CD automatizados (GitHub Actions + Webhooks de despliegue al servidor) requiere **4 horas adicionales**. En total: **1 jornada tecnica (6 a 7 horas)**, sin tiempo de inactividad para los usuarios finales.

3. ¿Se tendria que transformar completamente el proyecto o podemos generar la estructura sin afectaciones?
No se requiere una transformacion destructiva. La estructura desacoplada (frontend/ y backend/) ya esta preparada en el filesystem. Nginx continuara sirviendo la carpeta frontend/ y reenviando las llamadas /auth y /rest al contenedor backend/. El contrato de comunicacion HTTP se mantiene identico, garantizando cero riesgo de incompatibilidad.

4. Auditoria Granular del Proyecto y Plan de Mitigacion

Se ejecuto una revision estricta del codigo fuente, infraestructura de contenedores, contratos de datos y capas de red bajo cuatro perfiles de especialidad de ingenieria senior:

4.1 Perfil QA & Testing

Identificador	Hallazgo Detectado	Severidad	Estrategia de Mitigacion
QA-01	Ausencia de suite de pruebas automatizadas (unitarias y E2E) para el frontend.	ALTA	Implementar suite con Playwright / Jest para validar flujos criticos (creacion de RFC, aprobacion VOBO y filtros).
QA-02	Dependencia de mockeos en memoria (DB_CACHE) ante fallos temporales de red.	MEDIA	Incorporar mecanica de reintentos exponenciales con Circuit Breaker en cliente smartFetch.
QA-03	Riesgo de regresion en parseo de fechas internacionales e IST.	MEDIA	Crear tabla de fixtures de fechas (Excel y correos) con aserciones automatizadas en backend/tests/.

4.2 Perfil UX / UI (Experiencia e Interfaz de Usuario)

Identificador	Hallazgo Detectado	Severidad	Estrategia de Mitigacion
UX-01	Archivo index.html monolítico con mas de 29,000 lineas, dificultando mantenibilidad modular.	MEDIA	Modularizar en modulos ES (ESM) nativos por componentes (vistas, modales, utilerias).
UX-02	Manejo de picklists multiselect con listeners globales que pueden desincronizarse en recargas asincronas.	MEDIA	Estandarizar componentes de seleccion desacoplados con sincronizacion bidireccional mediante Custom Events.
UX-03	Contraste visual de badges de estado en modo oscuro bajo estandares WCAG 2.1 AA.	BAJA	Ajustar paleta de colores tokens CSS en :root para asegurar ratio de contraste mayor a 4.5:1.

4.3 Perfil CI / CD (Integracion y Despliegue Continuo)

Identificador	Hallazgo Detectado	Severidad	Estrategia de Mitigacion
CI-01	Despliegues manuales por SCP/SSH sin verificacion previa automatizada de sintaxis.	ALTA	Configurar GitHub Actions con verificacion obligatoria de linters (ESLint, Prettier) antes de merge.
CI-02	Falta de pipeline de construccion de imagenes Docker versionadas por tags de Git.	MEDIA	Construir imagenes de contenedor con versionado semantico (v3.0.x) y publicarlas en GitHub Packages.
CI-03	Ausencia de procedimiento de Rollback automatizado en caso de fallo post-despliegue.	MEDIA	Implementar script de rollback que restaure el commit previo y el snapshot de PostgreSQL.

4.4 Perfil Ciberseguridad (Defensa en Profundidad)

Identificador	Hallazgo Detectado	Severidad	Estrategia de Mitigacion
SEC-01	Archivos .env con credenciales locales susceptibles a ser comiteados por error.	CRITICA	Garantizar .gitignore riguroso en ambos repositorios, usar .env.example y almacenar secretos en GitHub Secrets.
SEC-02	Tokens JWT almacenados en localStorage expuestos a ataques XSS si se inyecta script.	MEDIA	Configurar cookies HttpOnly y SameSite=Strict para sesiones sensibles y cabeceras CSP estrictas en Nginx.
SEC-03	Falta de Rate Limiting granular en el endpoint de autenticacion /auth/v1/token.	MEDIA	Aplicar modulo express-rate-limit limitando a 5 intentos fallidos por IP cada 15 minutos.
SEC-04	Auditoria en PostgreSQL requiere retencion y rotacion de logs a largo plazo.	BAJA	Establecer politica de archivado mensual de audit_logs con pg_dump comprimido hacia almacenamiento cifrado.

5. Hoja de Ruta Priorizada de Mitigacion y Transicion

Para ejecutar la migracion y mitigar los hallazgos sin impacto en la operacion diaria, se establece el siguiente cronograma de ejecucion por fases:

Fase	Objetivo y Entregables	Plazo Estimado	Impacto Operativo
Fase 1	**Preparacion y Sanitizacion de Secretos:** Limpieza de credenciales en gitignore, creacion de .env.example limpios y creacion de repositorios en cuentas GitHub.	1 hora	Cero Impacto
Fase 2	**Push Inicial a GitHub:** Publicacion de Frontend (eaguilarh01) y Backend (eaguilarh), proteccion de ramas main y configuracion de Deploy Keys.	2 horas	Cero Impacto
Fase 3	**Automatizacion CI/CD y Hardening:** Incorporacion de rate-limiting en API Express, workflows de GitHub Actions para pruebas de sintaxis y webhook de despliegue.	4 horas	Cero Impacto
Fase 4	**Pruebas de Continuidad y Documentacion:** Verificacion de sincronizacion en vivo, validacion de permisos y cierre de memoria tecnica.	2 horas	Cero Impacto

DICTAMEN DE INGENIERIA:
La plataforma ATLAS-SAP Operaciones se encuentra en un estado funcional maduro y plenamente apta para la transicion a repositorios independientes de GitHub bajo la arquitectura planteada. La ejecucion no compromete la estabilidad del servicio On-Premise en Proxmox.